

Elimizde 4 ayrı panel var. Her panel için bu planı **ayrı ayrı** uygulayacaksın.

Panel	Dosya	Örnek Sayısı	Dikkat
MASTER	YARISMA_TRAIN_MASTER.csv	2.931	Ana veri seti
KANSER	YARISMA_TRAIN_KANSER.csv	388	Küçük
PAH	YARISMA_TRAIN_PAH.csv	372	Küçük + Çok dengesiz
CFTR	YARISMA_TRAIN_CFTR.csv	111	Çok küçük — özel kurallar var

Her panelde **353 kolon** var. Kolonlar şu gruplardan oluşuyor:

- AL_1 → AL_334 : Sayısal özellikler (popülasyon frekansları, in-silico skorlar vb.)
- CAT_1 → CAT_6 : Kategorik özellikler (popülasyon kaynağı, genotip vb.)
- EK_1 → EK_9 : Ek sayısal özellikler (SIFT, PolyPhen benzeri skorlar)
- AA_1, AA_2 : Aminoasit değişimi (örn: R → Q)
- Variant_ID : Varyant kimliği — modelde **kullanılmayacak**
- Label : Hedef değişken — 0 = Benign, 1 = Patojenik

AŞAMA 1 — VERİYİ TANI (EDA)

Adım 1.1 — Temel Yapıyı İncele

Her panel için şunları öğren ve bir yere kaydet:

- Kaç satır, kaç kolon var?
- Her kolonun veri tipi ne? (sayısal / metin)
- İlk birkaç satıra bak, değerler nasıl görünüyor?
- Kaç Patojenik (1), kaç Benign (0) örnek var? Yüzde olarak ne kadar?

Adım 1.2 — Kolon Gruplarını Belirle

AL_, CAT_, EK_, AA_ öneklerine göre kolon listelerini ayır. Her grubun kaç kolon içerdiğini not al.

Adım 1.3 — Eksik Veri Haritasını Çıkar

- Her kolon için eksiklik oranını (%) hesapla
- Sonuçları büyükten küçüğe sırala
- Eksiklik haritasını görsel olarak üret

Beklenen bulgular:

- Bazı AL_ kolonları %90+ eksik → bunları **Adım 2.1'de** sileceğiz
- CAT_6 neredeyse tüm panellerde %100 eksik → doğrudan silinecek
- Eksiklikler rastgele değil, gruplar halinde → imputasyon dikkatli yapılmalı

Kaydet: Eksiklik oranı > %85 olan kolonları bir listeye yaz. Bu listeye **Adım 2.1'de** geri dönülecek

Adım 1.4 — Sınıf Dengesini Gör

Her panel için Benign / Patojenik oranını bar grafiğiyle göster.

Bu oranlar modelin nasıl eğitileceğini belirleyecek. **Adım 6.1'de** bunlara geri dönülecek.

Adım 1.5 — Dağılım ve Korelasyon Grafikleri Üret

- EK_ kolonları için histogram çiz
 - Her sayısal feature ile Label arasındaki korelasyonu hesapla, büyükten küçüğe sırala
 - EK_ kolonları + Label için korelasyon heatmap'i üret
 - PCA ile veriyi 2 boyuta indir, Label'a göre renklendir: sınıflar görsel olarak ayrışıyor mu?
-

AŞAMA 2 — VERİ TEMİZLEME

Adım 2.1 — Kullanılmayacak Kolonları Çıkar

Şu kolonları veri setinden kaldır:

1. Variant_ID — kimlik kolonu, bilgi taşımaz
2. **Adım 1.3'te** kaydettiğin listeden: eksiklik oranı > %85 olan tüm kolonlar
3. CAT_6 — zaten neredeyse tamamen boş

Kaldırılan kolon sayısını not al.

Adım 2.2 — Kategorik Kolonları Sayıya Çevir

CAT_1, CAT_2 vb. kolonlarda gnomADe_EAS, AllofUs_EUR gibi metin değerler var. Bunları modele vermeden önce sayıya dönüştür.

- Önce eksik değerleri "UNKNOWN" gibi sabit bir değerle doldur
- Ardından her kategorik kolonu sayıya çevir, sayı temsili,nin hangi kategoriye temsil ettiğini not al.

Dikkat: CAT_3 ve CAT_4 kolonlarında C/C, T/T, G/G gibi genotip bilgisi var. Bu bilgi Label hakkında doğrudan ipucu verebilir. **Adım 3.1'de** leakage testi yapılacaktır.

Adım 2.3 — AA_1 ve AA_2'den Yeni Özellikler Üret

AA_1 ve AA_2 kolonlarında tek harfli aminoasit kodları var (örn: R, Q, L, P). Bu harfleri olduğu gibi modele verme. Bunlardan biyolojik anlam taşıyan sayısal özellikler türet:

Türetilecek özellikler:

- **Yük değişimi (charge_delta):** Referans ve alternatif aminoasitin elektrik yükü farkı. Pozitif → negatif geçiş ciddi fonksiyonel bozukluk işareti.
- **Hidrofobiklik farkı (hydro_delta):** Kyte-Doolittle skalasından türetilir. Büyük fark → yapısal bozukluk riski.
- **Aminoasit değişmedi mi? (aa_same):** AA_1 == AA_2 ise 1, değilse 0. Bu durum missense dışı bir mekanizmaya işaret edebilir.

İnternette aminoasit yük ve hidrofobiklik tablolarını araştırarak bu değerleri üret. AA_1 ve AA_2 ham kolonlarını bu adımın sonunda sil.

Neden önemli? Model ham harfleri yorumlayamaz ama sayısal farkları yorumlayabilir. Ayrıca bu feature'lar SHAP analizinde **Adım 10.1'de** biyolojik yorumlanabilirlik sağlar.

Adım 2.4 — EK_Kolonlarından Ek Özellikler Üret

EK_1 ve EK_2 iki farklı in-silico skoru temsil ediyor. Her ikisi de aynı yönde işaret ediyorsa güven artar:

- İki skurun **çarpımı** → kombine sinyal
- İki skurun **ortalaması** → konsensüs skoru

EK_9 negatif değer içerebilir (CADD skoru benzeri bir metrik olabilir). Olduğu gibi bırak, dönüşüm yapma.

Adım 2.5 — Eksiklik Bilgisini Özellik Olarak Kaydet

Bir feature'ın eksik olması da bilgidir. Örneğin bir varyantın belirli bir veri tabanında bulunmaması, o varyantın nadir olduğuna işaret edebilir.

Hâlâ eksik değer içeren her sayısal kolon için yeni bir binary kolon ekle: değer eksikse 1, doluyse 0.

Örnek: AL_7 kolonu eksikse → AL_7_missing = 1

AŞAMA 3 — DATA LEAKAGE ANALİZİ ▲

Leakage nedir? Model, tahmin yaparken gerçek dünyada bilmemesi gereken bir bilgiye erişiyorsa bu "veri sızıntısı"dır. Eğer Label'ı neredeyse doğrudan veren bir feature varsa ya da test seti bilgisi eğitime karışır, model gerçekte bir şey öğrenmez — sadece kopya çeker. Yarışmada bu tür bir model test setinde düşük skor alır.

Adım 3.1 — Target Leakage Testi

Her feature'ın Label ile ne kadar bilgi paylaştığını ölç. Bunun için **Mutual Information** yöntemini kullan.

- Eksik değerleri geçici olarak median ile doldur (sadece bu test için)
- Tüm feature'lar için MI skoru hesapla
- Büyükten küçüğe sırala ve ilk 20'yi incele

Sonuçtan ne anlayacaksın?

- MI skoru 1.0'a çok yakın bir feature varsa → o feature Label'ı neredeyse doğrudan veriyor → leakage şüphesi, o feature'ı çıkar
 - CAT_3, CAT_4 üst sıralarda mı? → leakage riski, bu kolonları veri setinden çıkarmayı düşün
 - EK_1, EK_2 üst sıralarda olmalı → bu beklenen ve normaldir
-

Adım 3.2 — Preprocessing Leakage Testi

Bu adımda kasıtlı olarak iki farklı pipeline kur ve sonuçları karşılaştıır:

Yanlış Pipeline: Veriyi train/test olarak ayırmadan önce imputer ve scaler'ı tüm veri üzerinde eğit. Modeli eğit, AUC'yi kaydet.

Doğru Pipeline: Önce train/test ayır. Imputer ve scaler'ı sadece train verisi üzerinde eğit, test verisine sadece transform uygula. Modeli eğit, AUC'yi kaydet.

İki AUC arasındaki farkı not al. Bu fark, leakage'ın performansı ne kadar yapay şişirdiğini gösterir.

Adım 3.3 — Pipeline Objesi Kullan (Leakage'ı Tamamen Önle)

Bundan sonraki tüm modelleriö Pipeline objesiyle kur. Bu yapı, cross-validation sırasında her fold'da preprocessing adımlarını otomatik olarak yalnızca training verisinde eğitir. Dışarıda hiçbir adım kalmamalı.

Sıra: Imputer → Scaler → Model

AŞAMA 4 — EKSİK VERİ YÖNETİMİ

Adım 4.1 — Hangi Kolonlara Ne Yapacağını Belirle

Adım 1.3'te kaydettiğin eksiklik listesini kullan. Kural şu:

Eksiklik Oranı	Yapılacak
> %85	Sil (Adım 2.1'de zaten yapıldı)
%20 ile %85 arası	İmputasyon + Adım 2.5'teki MissingIndicator
< %20	Sadece median imputasyon yeterli

Adım 4.2 — İmputasyon Yöntemlerini Karşılaştır

Üç farklı yöntemi aynı cross-validation ayarıyla test et, AUC skorlarını karşılaştır:

- **Median (SimpleImputer):** Küçük paneller için varsayılan.
- **KNN Imputer**
- **Iterative Imputer (MICE)**

Karar: En yüksek AUC veren yöntemi seç.

CFTR için: IterativeImputer kullanma. 111 örnekte overfit yapar. SimpleImputer(strategy="median") kullan.

AŞAMA 5 — AYKIRI DEĞER ANALİZİ

Adım 5.1 — Outlier Tespiti (Keşif Amaçlı)

EK_ kolonları için Z-score hesapla. $|Z| > 3$ olan değerleri say ve not et.

EK_9 kolonunun min ve max değerlerine özellikle bak — negatif değer görmek normal.

Adım 5.2 — Outlier Stratejisi

Biyolojik veri uyarısı: Genetik vericide outlier silme son derece risklidir. Nadir patojenik varyantlar tam da "aykırı" görünen bölgede yer alır. Bu yüzden **silme yerine sağlamlaştırma** yap.

Şu iki yaklaşımı dene ve karşılaştır:

- **RobustScaler:** IQR kullanarak scale eder, outlier'ları silmez ama etkisini azaltır. Pipeline'da StandardScaler yerine bunu kullan.
- **Winsorization:** Değerleri %1 ve %99 persentile sıkıştırır. Bilgi kaybı minimumdur.

Karar: RobustScaler'ı varsayılan olarak kullan. Gerçekten mantıksız değerler (negatif frekans gibi) varsa manuel olarak sil.

AŞAMA 6 — SINIF DENGESİZLİĞİ (CLASS IMBALANCE)

Adım 6.1 — Class Weight ile Baseline Al

Adım 1.4'te gördüğün dengesizliği hatırla. En basit ve en güvenli çözüm modele sınıf ağırlığı vermektir. `class_weight="balanced"` parametresiyle modeli eğit ve AUC, MCC, Recall değerlerini kaydet. Bu değerler senin baseline'ın.

Adım 6.2 — SMOTE ile Karşılaştır

CFTR paneli için bu adımı tamamen atla. 111 örnekten 21 benign var. SMOTE bu kadar az örnekten güvenilir sentetik veri üretemez.

SMOTE'u **mutlaka** `imblearn.pipeline.Pipeline` içinde kullan. Fold dışında uygulamak leakage yaratır (**Adım 3'e** bak).

Test et ve karşılaştır:

- Sadece `class_weight="balanced"`
- SMOTE + ağırlıksız model
- BorderlineSMOTE + ağırlıksız model

Hangisi daha yüksek Recall ve MCC veriyorsa onu kullan.

Adım 6.3 — SMOTE Kalite Testi

Eğer SMOTE kullanmaya karar verdiysen, sentetik verilerin gerçeğe benzeyip benzemediğini kontrol et.

A) Görsel test:

- Gerçek ve sentetik örnekleri ayrı etiketle
- PCA ile 2 boyuta indir
- İkisini aynı grafikte göster
- Beklenen: sentetik örnekler gerçeklerin arasına karışmalı, ayrı bir küme oluşturmamalı

B) İstatistiksel test:

- Her feature için Kolmogorov-Smirnov (KS) testi uygula
- $p > 0.05 \rightarrow$ dağılımlar benzer $\rightarrow$ iyi
- $p < 0.05 \rightarrow$ dağılımlar farklı $\rightarrow$ o feature için SMOTE kalitesi düşük

Eğer birçok feature'da $p < 0.05$ çıkıyorsa bu panelde SMOTE kullanma, class_weight yeterli.

AŞAMA 7 — FEATURE SELECTION (ÖZELLİK SEÇİMİ)

Adım 7.1 — Sabit Değerli Kolonları Çıkar

Neredeyse tüm örneklerde aynı değeri alan kolonlar sınıflandırmaya katkı sağlamaz. Varyans eşiği belirle ve bu eşiğin altındaki kolonları çıkar.

Adım 7.2 — MI Skorlarına Göre En İyi Feature'ları Seç

Adım 3.1'de hesapladığın MI skorlarını kullan. Belirli bir eşiğin üzerindeki feature'ları seç.

- MASTER ve KANSER için: üst %50
- PAH ve CFTR için: üst %30 (daha az feature, daha az overfit riski)

Seçilen feature sayısını not al.

Adım 7.3 — LightGBM ile Feature Importance

Hızlı bir LightGBM modeli eğit ve feature importance değerlerini al. En önemli 30 feature'ı listele ve bar grafiğiyle göster.

Bu liste ile Adım 7.2'nin listesini karşılaştı. İkisinde de üst sıralarda olan feature'lar en güvenilir adaylar.

AŞAMA 8 — MODEL EĞİTİMİ

Adım 8.1 — Final Pipeline Yapısını Kur

Adım 3.3'te öğrenilen Pipeline yapısını kullan. Sıra:

Imputer $\rightarrow$ RobustScaler $\rightarrow$ (SMOTE varsa) $\rightarrow$ Model

SMOTE varsa `sklearn.pipeline.Pipeline` yerine `imblearn.pipeline.Pipeline` kullan.

Adım 8.2 — Modelleri Sırayla Test Et

Her modeli aynı cross-validation ayarıyla (**Adım 8.3'teki** strateji) test et. Sonuçları karşılaştır:

Sıra	Model	Not
1	Logistic Regression	Baseline. Her zaman ilk bunu dene.
2	Random Forest	Güçlü ve yorumlanabilir.
3	LightGBM	Ana model adayı. Eksik değerlere toleranslı.
4	XGBoost	LightGBM ile karşılaştır.

CFTR için: Sadece Logistic Regression ve Random Forest dene. LightGBM'i çok kısıtlı parametrelerle dene (sığ ağaç, yüksek regularizasyon). Derin öğrenme modeli kullanma.

Her model için şu metrikleri hesapla ve tabloya gir: ROC-AUC, PR-AUC, MCC, Recall, F1

Adım 8.3 — Validation Stratejisi

Her model için şu CV yapısını kullan:

- MASTER, KANSER, PAH: StratifiedKFold(n_splits=10)
 - CFTR: RepeatedStratifiedKFold(n_splits=5, n_repeats=10) → 50 farklı fold kombinasyonu, küçük veri için daha güvenilir
-

Adım 8.4 — Threshold Optimizasyonu

Default eşik 0.5 yerine, **Recall $\geq$ 0.90** koşulunu sağlayan en yüksek Precision'ı veren eşiği bul.

precision_recall_curve fonksiyonu farklı threshold değerlerinde Precision ve Recall ikililerini verir. Bu ikilileri tara ve koşulu sağlayan en iyi threshold'u kaydet.

Her panel için bu değer farklı olacak.

AŞAMA 9 — PERFORMANS DEĞERLENDİRME

Adım 9.1 — Metrikleri Hesapla

Adım 8.4'te bulduğun threshold ile tahminleri üret. Şu metriklerin hepsini hesapla:

Metrik	Neden önemli?
ROC-AUC	Genel ayırt edicilik gücü
PR-AUC	Dengesiz veri için ROC-AUC'den güvenilir
MCC	Tüm hata tiplerini dengeli değerlendirir
Recall (Sensitivity)	En kritik. Patojenik atlama oranı.
Specificity	Benign doğru tespit. Gereksiz müdahaleyi önler.
F1	Precision-Recall dengesi

Neden Recall en önemli? Patojenik bir varyantı "Benign" olarak işaretlemek, hastanın tedaviden mahrum kalması demektir.

Adım 9.2 — Sonuçları Tabloya Dök

Her panel ve her model için ayrı satır:

Panel	Model	AUC	PR-AUC	MCC	Recall	Threshold
MASTER	Logistic Reg.	?	?	?	?	?
MASTER	LightGBM	?	?	?	?	?
KANSER	LightGBM	?	?	?	?	?
PAH	LightGBM	?	?	?	?	?
CFTR	Random Forest	?	?	?	?	?

AŞAMA 10 — AÇIKLANABİLİRLİK (SHAP)

Adım 10.1 — SHAP Analizi

Her panel için en iyi modelin SHAP değerlerini hesapla.

- **Summary Plot:** Hangi feature'lar en çok etkiliyor? Yüksek değeri olan feature patojenisite mi artırıyor azaltıyor mu?
- **Force Plot:** Tek bir örnek için kararı açıkla. Özellikle yanlış sınıflandırılan örnekler için bak.

Kontrol et:

- EK_1, EK_2 üst sıralarda mı? → Beklenen, normal.
- **Adım 2.3'te** ürettiğin charge_delta, hydro_delta listede mi? → Biyolojik anlamlılık doğrulandı.
- CAT_3 veya CAT_4 en üstte mi? → Leakage şüphesi! **Adım 3.1'e** geri dön, bu kolonları çıkar ve modeli yeniden eğit.

AŞAMA 11 — HATA ANALİZİ

Adım 11.1 — Yanlış Sınıflandırılan Örneklerle Bak

- **False Negative'leri** bul: Patojenik iken Benign tahmin edilenler. Bunların tahmin olasılıkları ne kadar? 0.5'e yakınsa model kararsız kalmış; 0.9 gibi yüksek bir güvenle yanlış yaptıysa bu daha ciddi bir hata.
- **False Positive'leri** bul: Benign iken Patojenik tahmin edilenler.
- Her iki gruptaki örnek sayısını ve tahmin olasılıklarının dağılımını belgele.

Kütüphane: pandas, matplotlib

Adım 11.2 — Belirsiz Bölgeyi Tespit Et

Tahmin olasılığı 0.4 ile 0.6 arasında kalan örnekler modelin en az emin olduğu örneklerdir. Bunların sayısını ve içlerinde kaç Patojenik olduğunu raporla. Bu örnekler gerçek klinik pratikte de zaten belirsiz kalır.

AŞAMA 12 — PANEL BAZLI ÖZEL NOTLAR

CFTR Paneli (111 örnek) için Ekstra Kurallar

1. **Adım 7.2:** Feature selection'ı çok agresif yap. Üst %30'u seç.
2. **Adım 8.2:** Sadece Logistic Regression ve Random Forest. LightGBM'i sığ ağaçla ve yüksek regularizasyonla dene.
3. **Adım 6.2:** Bu adımı tamamen atla. Sadece class_weight="balanced" kullan.
4. **Adım 8.3:** RepeatedStratifiedKFold kullan.
5. Metrikler yüksek varyansa sahip olacak — bu normaldir, yorumlarken ihtiyatlı ol.

KONTROL LİSTESİ — Her Panel Tamamlanınca İşaretle

- ☐ Adım 1: EDA tamamlandı, eksiklik ve sınıf dağılımı kaydedildi
- ☐ Adım 2.1: Yüksek eksikli kolonlar silindi
- ☐ Adım 2.2: Kategorik kolonlar encode edildi
- ☐ Adım 2.3: AA_1/AA_2'den biyolojik feature'lar üretildi
- ☐ Adım 2.4: EK_ kombinasyon feature'ları eklendi
- ☐ Adım 2.5: MissingIndicator binary feature'ları eklendi
- ☐ Adım 3.1: Leakage audit yapıldı, şüpheli feature'lar incelendi
- ☐ Adım 3.2: Hatalı vs doğru pipeline AUC farkı ölçüldü
- ☐ Adım 3.3: Tüm modeller Pipeline objesiyle kuruldu
- ☐ Adım 4.2: En iyi imputasyon yöntemi seçildi
- ☐ Adım 5: RobustScaler kullanılıyor, outlier silinmedi
- ☐ Adım 6.1: class_weight="balanced" ile baseline alındı
- ☐ Adım 6.2: SMOTE karşılaştırması yapıldı (CFTR hariç)
- ☐ Adım 6.3: SMOTE kalitesi görsel ve istatistiksel test edildi
- ☐ Adım 7: Feature selection tamamlandı, seçilen kolon sayısı kaydedildi
- ☐ Adım 8.2: En az 3 model test edildi
- ☐ Adım 8.4: Recall ≥ 0.90 için threshold belirlendi
- ☐ Adım 9.2: Tüm metrikler paneller arası karşılaştırma tablosuna girildi
- ☐ Adım 10: SHAP grafikleri üretildi, önemli feature'lar doğrulandı
- ☐ Adım 11: False Negative'ler incelendi ve raporlandı